Chapter 7: Normalization

The goal of schema normalization is to minimize data redundancy in database.

Outline

- Data redundancy
- Functional dependency
- Decomposition
- Schema normalization

7.1 Data Redundancy

- A bad design may result in repeated information and data inconsistency among the various copies of information.

Borrow

StuID	Name	Dept	Mngr	BookID	Title	Date
S0001	Alice	CS	Boss1	B001	Book1	2010-1-1
S0001	Alice	CS	Boss1	B002	Book2	2010-1-2
S0001	Alice	CS	Boss1	B003	Book3	2010-1-3
S0002	Bob	CS	Boss1	B001	Book1	2010-1-5
S0003	Carol	ME	Boss2	B003	Book3	2011-2-6

There is repetition of information
Need to use null values (if we add a new student with no book borrowing)
Change of manager of CS dept leads to data inconsistency

7.2 Decomposition

- The only way to avoid the repetition-of-information problem in the Borrow schema is to decompose it into smaller schemas
- Let *R* be a schema of relation r. If the schemas *R*1 and *R*2 form a **decomposition** of *R*, then *R* = *R*1 ∪ *R*2.
  - R = Borrow (StuID, Name, Dept, Mngr, BookID, Title, Date)
  - R1 = Student (StuID, Name, Dept)
  - R2 = Department (Dept, Mngr)
  - R3 = Book (BookID, Title)
  - R4 = Loan (StuID, BookID, Date)

Decomposition

- Not all decompositions are good. Suppose we decompose

  employee(ID, name, street, city, salary)

  into

  employee1 (*ID*, *name*)

  employee2 (*name*, *street, city, salary*)

  The problem arises when we have two employees with the same name

- The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**.

Lossless Decomposition

- Let *R* be a relation schema and let R_1 and R_2 form a decomposition of R . That is R = R_1 U R_2
- We say that the decomposition is a **lossless decomposition** if there is no loss of information by replacing R with the two relation schemas R_1 U R_2

$$\prod_{R_1}(r) \bowtie \prod_{R_2}(r) = r$$

- Conversely a decomposition is lossy if

$$r \subset \prod_{R_1}(r) \bowtie \prod_{R_2}(r)$$

Lossy Decomposition

ID	name	street	city	salary
57766	Kim	Main	Perryridge	75000
98776	Kim	North	Hampton	67000

employee

ID	name
57766	Kim
98776	Kim

name	street	city	salary
Kim	Main	Perryridge	75000
Kim	North	Hampton	67000

natural join

ID	name	street	city	salary
57766	Kim	Main	Perryridge	75000
57766	Kim	North	Hampton	67000
98776	Kim	Main	Perryridge	75000
98776	Kim	North	Hampton	67000

$$r \subset \prod_{R_1}(r) \bowtie \prod_{R_2}(r)$$

Join on **common** attributes with the same values

Example of Lossless Decomposition

- Decomposition of $R = (A, B, C)$

  $R_1 = (A, B)$ $\quad$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$$\Pi_A(r) \bowtie \Pi_B(r) = r$$

A	B	C
α	1	A
β	2	B

Example of Lossless Decomposition

Borrow

StuID	Name	Dept	Mngr	BookID	Title	Date
S0001	Alice	CS	Boss1	B001	Book1	2010-1-1
S0001	Alice	CS	Boss1	B002	Book2	2010-1-2
S0001	Alice	CS	Boss1	B003	Book3	2010-1-3
S0002	Bob	CS	Boss1	B001	Book1	2010-1-5
S0003	Carol	ME	Boss2	B003	Book3	2011-2-6

Decomposed

StuID	Name	Dept	Mngr	BookID	Date
S0001	Alice	CS	Boss1	B001	2010-1-1
S0001	Alice	CS	Boss1	B002	2010-1-2
S0001	Alice	CS	Boss1	B003	2010-1-3
S0002	Bob	CS	Boss1	B001	2010-1-5
S0003	Carol	ME	Boss2	B003	2011-2-6

Borrow1

BookID	Title
B001	Book1
B002	Book2
B003	Book3

Book

Composed

StuID	Name	Dept	Mngr	BookID	Title	Date
S0001	Alice	CS	Boss1	B001	Book1	2010-1-1
S0001	Alice	CS	Boss1	B002	Book2	2010-1-2
S0001	Alice	CS	Boss1	B003	Book3	2010-1-3
S0002	Bob	CS	Boss1	B001	Book1	2010-1-5
S0003	Carol	ME	Boss2	B003	Book3	2011-2-6

Normalization Theory

- Decide whether a particular relation *R* is in "good" form.
- In the case that a relation *R* is not in "good" form, decompose it into set of relations {R_1, R_2, ..., R_n} such that
  - Each relation is in good form
  - The decomposition is a lossless decomposition
- The theory is based on:
  - Functional dependencies

Functional Dependencies

- There are usually a variety of constraints (rules) on the data in the real world.
- An instance of a relation that satisfies all such real-world constraints is called a **legal instance** of the relation;
- For example:
  - Students and instructors are uniquely identified by their ID.
  - Each student and instructor has only one name.
  - Each instructor and student is officially associated with only one department.
  - Each department has only one value for its budget, and only one associated building.

7.3 Functional Dependency

- In a given relation, an attribute Y is functionally dependent on a set of attributes X (written as X → Y), if and only if each X value is associated with precisely one Y value
- The value for a certain set of attributes determines uniquely the value for another set of attributes.
- A generalization of the notion of a *key.*
- Borrow(StuID, Name, Dept, Mngr, BookID, Title, Date)
  - StuID → Name; StuID → Dept
  - Dept → Mngr; StuID → Mngr
  - BookID → Title; (StuID, BookID) → Date
- Note: In a relation, all attributes are functionally dependent on candidate keys, and also superkeys. For examples:
  - (StuID, BookID) → Name
  - (StuID, BookID, Title) → Name

Functional Dependencies Definition

- Let R be a relation schema

$$\alpha \subseteq R \quad and \quad \beta \subseteq R$$

- The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on R if and only if

for any legal relations r(R), whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β.

$$t_1[\alpha] = t_2[\alpha] \quad \Rightarrow \quad t_1[\beta] = t_2[\beta]$$

That is, each α value is associated with precisely one β value

- Example: Consider r(A,B) with the following instance of r.

A	B
1	4
1	5
3	7

- On this instance, $B \rightarrow A$ hold; $A \rightarrow B$ does **NOT** hold,

Keys and Functional Dependencies

- K is a superkey for relation schema R, if and only if $K \rightarrow R$
- K is a candidate key for R, if and only if
  - $K \rightarrow R$, and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

  in_dep (*ID, name, salary, dept_name, building, budget*).

  We expect these functional dependencies to hold:

  dept_name→ *building*

  ID → *building*

  but would not expect the following to hold:

  dept_name → *salary*

Use of Functional Dependencies

- We use functional dependencies to:
  - To test relations to see if they are legal under a given set of functional dependencies.
    - If a relation *r* is legal under a set *F* of functional dependencies, we say that *r* **satisfies** *F.*
  - To specify constraints on the set of legal relations
    - We say that *F* **holds on** *R* if all legal relations on *R* satisfy the set of functional dependencies *F.*
- Note: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances.
  - For example, a specific instance of *instructor* may, by chance, satisfy *name* $\rightarrow$ *ID.*

Trivial Functional Dependency

- A trivial functional dependency is a functional dependency of an attribute on a superset of itself
  - (StuID, BookID) → StuID
  - Name → Name
- In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

- A trivial functional dependency does not provide meaningful information, so in the following we always talk about functional dependencies that are not trivial

Full Functional Dependency

- An attribute is fully functionally dependent on a set of attributes X if it is:
  - functionally dependent on X, and
  - not functionally dependent on any proper subset of X
- Full functional dependency
  - StuID → Name
  - StuID → Dept
  - (StuID, BookID) → Date
  - BookID → Title
- Partial functional dependency
  - (StuID, BookID) → Name (as there is also StuID → Name)
  - (StuID, BookID) → Title
  - (StuID, Name) → Dept

Transitive functional dependency:

- A transitive functional dependency is an indirect functional dependency
- If X → Y, Y → Z, then X → Z
- If StuID → Dept, Dept → Mngr, then StuID → Mngr is a transitive functional dependency

Prime Attributes and Non-prime Attributes

- Prime attribute: A prime attribute is an attribute that occurs in a **candidate** key.
- Non-prime attribute: A non-prime attribute is an attribute that does not occur in any candidate key
- In Borrow(StuID, Name, Dept, Mngr, BookID, Title, Date) relation,
  - *StuID* is a prime attribute
  - *Dept* is not a prime attribute

7.4 Schema normalization

- The goal of schema normalization is to minimize data redundancy in database. This is accomplished by designing relation schemas that are in an appropriate **normal form**.

Normal Forms

- The normal forms provide criteria for determining if a given relation schema is in “good form” . The common normal forms used in normalization are:
  - 1NF, 2NF, 3NF, BCNF
- Normalization is a process to design the database schema up to higher normal forms, which are usually 3NF or BCNF.
- If a given relation schema is not in “good form,” then we decompose it into a number of smaller relation schemas, each of which is in an appropriate normal form. The decomposition must be a lossless decomposition.

1st Normal Form (1NF)

- 1NF is the basic requirement for a relational schema:
  - Each attribute value contains only one simple value, i.e., the domains of all attributes of R are atomic
  - There are no duplicate tuples (i.e. there is a primary key)
- Example:
  - Borrow (StuID, Name, Dept, Mngr, BookID, Title, Date) $\in$ 1NF

StuID	Name	Dept	Mngr	BookID	Title	Date
S0001	Alice	CS	Boss1	B001	Book1	2010-1-1
S0001	Alice	CS	Boss1	B002	Book2	2010-1-2
S0001	Alice	CS	Boss1	B003	Book3	2010-1-3
S0002	Bob	CS	Boss1	B001	Book1	2010-1-5
S0003	Carol	ME	Boss2	B003	Book3	2011-2-6

Borrow

2nd Normal Form (2NF)

- 2NF: If a relational schema R∈1NF, and every **non-prime attribute** in R is fully functionally dependent on every candidate key of R, then R∈2NF
- 2NF means that every non-prime attribute in R is not partially functionally dependent on any candidate key.
- Example:
  - Borrow (StuID, Name, Dept, Mngr, BookID, Date) ∉ 2NF
    - StuID → Name
    - StuID → Dept
    - Dept → Mngr
    - StuID → Mngr
    - BookID → Title
    - (StuID, BookID) → Date
  - *Name, Dept, Title and Mngr* are partially functionally dependent on the candidate key

Normalize to 2NF

- To normalize a schema to 2NF, decompose the schema using the set of **prime** attributes on which some non-prime attributes depend. The set of the prime attributes used for decomposition is not a candidate key or a superkey.
- Example: Normalize the schema Borrow (StuID, Name, Dept, Mngr, BookID, Title, Date) to 2NF
  - Decompose the schema **Borrow** using StuID, into **Student** and **Borrow1**
    - **Student (StuID, Name, Dept, Mngr) ∈ 2NF**
      - StuID → Name; StuID → Dept; StuID → Mngr
    - **Borrow1 (StuID, BookID, Title, Date) ∉ 2NF**
      - (StuID, BookID) → Title; BookID → Title

StuID	Name	Dept	Mngr
S0001	Alice	CS	Boss1
S0002	Bob	CS	Boss1
S0003	Carol	ME	Boss2

Student

StuID	BookID	Title	Date
S0001	B001	Book1	2010-1-1
S0001	B002	Book2	2010-1-2
S0001	B003	Book3	2010-1-3
S0002	B001	Book1	2010-1-5
S0003	B003	Book3	2011-2-6

Borrow1

Normalize to 2NF

- To normalize a schema to 2NF, decompose the schema using the set of **prime** attributes on which some non-prime attributes depend. The set of the prime attributes used for decomposition is not a candidate key or a superkey.
- Example: Normalize the schema Borrow (StuID, Name, Dept, Mngr, BookID, Title, Date) to 2NF
  - Decompose the schema Borrow using StuID
    - Student (StuID, Name, Dept, Mngr) $\in$ 2NF
    - Borrow1 (StuID, BookID, Title, Date) $\notin$ 2NF
  - **Further Decompose the schema Borrow1 using BookID**
    - Book (BookID, Title) $\in$ 2NF
    - Borrow2 (StuID, BookID, Date) $\in$ 2NF
  - 2NF does not completely solve data redundancy

StuID	Name	Dept	Mngr
S0001	Alice	CS	Boss1
S0002	Bob	CS	Boss1
S0003	Carol	ME	Boss2

Student

BookID	Title
B001	Book1
B002	Book2
B003	Book3

Book

StuID	BookID	Date
S0001	B001	2010-1-1
S0001	B002	2010-1-2
S0001	B003	2010-1-3
S0002	B001	2010-1-5
S0003	B003	2011-2-6

Borrow2

3rd Normal Form (3NF)

- 3NF: If a relational schema R∈1NF, and every non-prime attribute is not transitively functionally dependent on any candidate key in R, then R∈3NF.
- Example:
  - Student(<u>StuID</u>, Name, Dept, Mngr) ∈ 2NF
  - Student(<u>StuID</u>, Name, Dept, Mngr) ∉ 3NF
  - Functional dependencies:
    - StuID → Dept, Dept → Mngr
    - So StuID → Mngr is a transitively functional dependency

Normalize to 3NF

- To normalize a schema to 3NF, decompose the schema using the set of attributes on which some non-prime attributes depend. The set of attributes used for decomposition is not a candidate key or superkey
- Example: Normalize Student(StuID, Name, Dept, Mngr) to 3NF
  - Student2(StuID, Name, Dept) $\in$ 3NF
  - Department(Dept, Mngr) $\in$ 3NF
- 3NF means that every non-prime attribute in R is functionally dependent on and only on a candidate key or superkey. Thus If R$\in$3NF, then R$\in$2NF.

Student2

StuID	Name	Dept
S0001	Alice	CS
S0002	Bob	CS
S0003	Carol	ME

Department

Dept	Mngr
CS	Boss1
ME	Boss2

Boyce-Codd Normal Form (BCNF)

- BCNF: If relational schema $R \in 1NF$, and for every functional dependency $X \rightarrow Y$, X is a candidate key, or a superkey, then $R \in BCNF$
- BCNF means that every attribute in R is functionally dependent on and only on a candidate key or superkey. Thus if $R \in BCNF$, then $R \in 3NF$.
- Only in rare cases $R \in 3NF$ but $R \notin BCNF$

Boyce-Codd Normal Form (Cont.)

- Example schema that is ***not*** in BCNF:

  in_dep (*ID, name, salary, dept_name, building, budget*)

  because :

  - *dept_name*→ *building, budget*
  - *dept_name* is not a superkey nor a candidate key
- Decompose ***in_dept*** into ***instructor*** and ***department***
  - ***instructor*** (*ID, name, salary, dept_name*) is in BCNF
  - ***department*** *(dept_name, building, budget*) is in BCNF

Boyce-Codd Normal Form (Cont.)

- Only in rare cases R∈3NF but R∉BCNF
- Consider a schema:

  dept_advisor(s_ID, i_ID, dept_name)
- With function dependencies:

  $i\_ID \rightarrow dept\_name$; $s\_ID, dept\_name \rightarrow i\_ID$
- Two candidate keys = {*s_ID, dept_name*}, {*s_ID, i_ID* }
- *dept_advisor* is in 3NF
  - *none of s_ID, dept_name, i_ID are non-prime attributes*
- However, *dept_advisor* is not in BCNF
  - *i_ID* is not a candidate key or superkey.
- **BCNF prevents a prime attribute to be partially or transitively dependent on a candidate key.**

s_ID	i_ID	dept_name
S0001	I0001	CS
S0001	I0002	ME
S0003	I0002	ME